

# OpenCode × iagent/standard

## 对齐 Claude Code 的 Harness 改造方案

基于 2026 年 Coding Agent Harness 论文、公开 Benchmark、OpenCode 源码与内部链路约束的批判性分析

### 先给结论：能，但必须限定比较口径。

如果 Claude Code 与 OpenCode 都通过同一 IAgent 链路使用同一个 iagent/standard 后端，并在相同任务、工具、权限、时间与 Token 预算下比较，那么通过针对 Harness 的定向改造，OpenCode 达到“统计意义上的能力持平”是现实目标，工程可行性为中高。

但如果“持平”指 OpenCode + iagent/standard 全面追平 Claude Code + 最新原生 Claude/Opus、覆盖所有仓库、所有 Agent Teams/IDE/长上下文/安全与企业能力，则不应作为近期承诺：这是移动靶，而且模型-运行时协同优化无法完全靠 OpenCode 源码补偿。

**建议项目目标：不是“复制 Claude Code”，而是“在公司真实 Coding Workload 上建立可证明的非劣或 Pareto 优势”。**

版本：2026-08-12

## 0. 执行摘要：到底能不能？

| 问题                                                               | 结论       | 原因                                                               |
|------------------------------------------------------------------|----------|------------------------------------------------------------------|
| OpenCode + iagent/standard 能否与 Claude Code + iagent/standard 持平？ | 能，目标合理   | 同模型后，Harness 成为主要变量；公开研究显示 Harness 可造成 27pp 级成功率差异及 40× 级效率差异。   |
| 能否“全面超过” Claude Code？                                            | 近期不建议承诺  | Claude Code 是快速移动、闭源且与 Claude 模型协同优化的目标；全维度能力无法被单一 benchmark 证明。 |
| 是否应该大改 OpenCode Core？                                            | 不应该一开始就做 | 应先用插件/Feature Flag 验证 hypothesis，再把被 A/B 证明有效的策略下沉到 Core。        |

**最重要的战略判断：**如果 Claude Code 也经 Anthropic→OpenAI 代理落到 iagent/standard，那么 Claude Code 原本最强的“Claude 模型 + Claude Code 原生 Harness 联合优化”优势会被削弱。此时你们真正比较的是两个 Harness 如何驱动同一个模型。对 OpenCode 来说，这是一个明显更可赢的战场。

**最重要的工程判断：**由于内部 iagent/standard 链路的短请求端到端延迟明显高于模型直连，因此你们应该优先优化“每个任务需要多少次模型往返、每次携带多少上下文、是否存在空转/重复搜索”，而不是优先增加更多 Agent。多 Agent 很可能把高延迟放大。

## 1. 为什么“能”：Harness 确实可以把同一个模型拉开很大差距

- **证据 1：Claw-SWE-Bench：**固定模型时，Harness 选择可带来约 27.4 个百分点的 Pass@1 差异；同一 GLM 5.1，从 minimal adapter 19.1% 到 full adapter 73.4%。这说明“改 Harness”并非微调，而可能达到换模型级别的收益。[R3]
- **证据 2：The Scaffold Effect：**同一模型跨 Goose/OpenCode/OpenHands-SDK，成功率差距并不总是巨大，但每个 solved task 的 Token 消耗可相差约 40×；OpenCode 的典型失败指纹包括 idle-loop/TIME。说明 OpenCode 存在可直接优化的运行时浪费。[R1]
- **证据 3：Harness-Bench：**5,194 条执行轨迹显示，能力应该以 model-harness configuration 为单位报告；Harness 会改变完成率、过程质量、效率和失败行为。[R2]
- **证据 4：公开 Coding Agent Index：**Artificial Analysis 已经按“同一 Opus 模型、不同 Harness”展示 OpenCode、Claude Code 等差异，并同时记录成功率、Token、成本、wall time。说明行业已经从“只比模型”转向“比模型×Harness”。[R4]

**推论：**在“同一个 iagent/standard”这一约束下，OpenCode 与 Claude Code 之间的差距并不是不可触碰的模型差距，而是可以被拆成搜索策略、上下文工程、工具协议、验证策略、停止策略、错误恢复等可测量的 Harness 差距。

## 2. 为什么“不能”把目标写成“全面追平 Claude Code”

- 1. **Claude Code 是移动靶：**Claude Code 当前已具备 subagents、agent teams、dynamic workflows、hooks、plugins、自动 compaction、memory 等能力，并持续高频演进。你们若按功能列表追赶，会长期落入 feature chasing。[R6][R7][R8]
- 2. **原生 Claude 场景仍有模型-Harness 协同优势：**Claude Code 的系统提示、工具设计、上下文与模型行为是围绕 Claude 共同优化的。若竞争对象改成 Claude Code + 原生 Opus，而不是 Claude Code + iagent/standard，难度会陡升。
- 3. **Harness 无法突破模型能力上限：**Harness 可以显著改善探索、执行、恢复和效率，但不能保证把较弱的基础模型变成在所有深推理/架构问题上等价于更强模型。
- 4. **不存在一个“总代表分数”：**SWE-bench、Terminal-Bench、repo Q&A、长周期演进、安全边界、用户干预率评估的是不同能力。单一 leaderboard 分数不能证明“综合持平”。
- 5. **源码改造存在非线性连锁反应：**System prompt、tool schema、tool output、context、compaction、stop policy 是闭环系统。一个看似局部的改动可能改变后续工具选择、上下文增长和停止时机，因此必须用 Feature Flag + ablation 控制风险。

## 3. iagent/standard 场景的特殊性：这是你们最重要的“局部优势”

本报告按你们已提供的内部环境信息：iagent/standard 是 IAgent Gateway 的标准模型别名；OpenCode 走 OpenAI-compatible 接口，Claude Code 可通过 Anthropic Messages → OpenAI Chat 的代理转换后落到同一后端。内部测试还显示，该链路的端到端响应延迟明显高于模型直连。

这意味着评测应明确分成两个基线：

| 基线                                     | 用途             | 是否作为“持平”目标             |
|----------------------------------------|----------------|------------------------|
| A. Claude Code + iagent/standard       | 同模型 Harness 基线 | 是，主要验收基线               |
| B. OpenCode Base + iagent/standard     | 当前差距基线         | 是                      |
| C. Modified OpenCode + iagent/standard | 改造目标           | 是                      |
| D. Claude Code + 原生最新 Claude/Opus      | 能力上限参考         | 否，只做 ceiling reference |

**关键推论 1：高延迟链路让“减少模型往返次数”比“增加并发 Agent”更值钱。**如果一次额外模型 turn 的端到端等待本身就很重，那么 5 次无意义的重复搜索/空转，可能比一次局部代码优化造成更大的用户体验损失。不要假设并行总能抵消：并行会同时增加请求量、Rate Limit 压力和上下文汇总成本。

**关键推论 2：同一 session 不应随意切换模型。**你们此前已经观察到不同后端在 tool call、JSON、thinking、压缩、system prompt 遵循、patch 格式方面存在差异。为保证可识别实验，核心 benchmark 应固定 iagent/standard，一个 session 内不做模型切换。

## 4. 第一个必须验证的源码级假设：iagent/standard 是否落到了错误/过于通用的 System Prompt

OpenCode 当前 `packages/opencode/src/session/system.ts` 会依据 `model.api.id` 的字符串特征选择 provider/model-family prompt：GPT、Codex、Gemini、Claude、Kimi 等走不同 prompt，否则落到默认 prompt。[R9]

| 检查项                                                             | 为什么重要                                                                                   |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| OpenCode 内部 `model.api.id` 到底看到的是 `iagent/standard`，还是底层真实模型家族？ | 如果只看到 alias，可能无法命中针对模型家族调优的 prompt，直接进入 PROMPT_DEFAULT。                                 |
| IAgent Gateway 是否能额外返回 model capability metadata？               | 比字符串猜测更可靠：tool call、reasoning、context、patch preference、parallel tool call 等应显式建模。       |
| Claude Code→代理路径是否做了额外 prompt/tool schema 转换？                   | 如果做了，这部分可能正是 Claude Code + iagent/standard 的隐性优势，需要 trace diff 才能看见。                    |
| OpenCode 自定义 agent prompt 是 additive 还是 replacement？            | OpenCode 历史上不同 provider 路径出现过 prompt layering 不一致的问题；错误覆盖 base prompt 会显著伤害行为。<br>[R10] |

**建议：**不要直接给 iagent/standard 硬编码一个巨大 prompt。先新增 `ModelCapabilityProfile`，让 gateway/配置显式告诉 OpenCode：模型家族、上下文、tool-call 能力、是否偏好 dedicated file tools、是否支持并行工具调用、是否适合长 reasoning。System prompt 只消费 capability profile，而不是靠 alias 字符串猜。

## 5. 改造路线：按收益/风险排序，而不是按“功能酷炫度”排序

| 优先级 | 改造模块                        | 核心动作                                                                                | 预期收益                    | 连锁风险 | 是否先用插件验证 |
|-----|-----------------------------|-------------------------------------------------------------------------------------|-------------------------|------|----------|
| P0  | Trace/Benchmark             | 完整记录每 turn 的 prompt token、tool、tool output、context、latency、finish reason、compaction | 让所有后续改造可归因              | 低    | 不需要改行为   |
| P1  | Model Profile/System Prompt | 为 iagent/standard 建 capability profile；修正 base prompt layering                      | 高，可能立刻改善 tool/search 行为 | 中    | 是        |
| P1  | Turn/Stop Policy            | no-op/stagnation detector；重复 action 抑制；明确完成门槛                                       | 高，尤其适合高延迟 Gateway       | 中    | 是        |

|    |                   |                                                              |                     |   |    |
|----|-------------------|--------------------------------------------------------------|---------------------|---|----|
| P1 | Repo Exploration  | Search-first; grep/LSP 优先; 限制顺序 Read; 相关性排序                  | 高, 减少无关 context     | 中 | 是  |
| P1 | Context Governor  | tool output 截断/摘要; log 限额; 主动 compaction; 保留 evidence ledger | 高, 降低 token/latency | 高 | 部分 |
| P2 | Verification Gate | affected tests → lint/typecheck → diff review → acceptance   | 高, 提高一次成功率          | 中 | 是  |
| P2 | Tool/Edit 协议      | 优化 edit/patch 失败恢复; 统一 dedicated tools 与 bash 边界             | 中高                  | 中 | 是  |
| P3 | Minimal Subagent  | 只给 Explore/Review 两类隔离上下文, 默认 0-2 个                          | 仅宽任务收益高             | 高 | 是  |

6. P0：先建立 Agent Trajectory Profiler，否则所有改造都不可证伪

第一阶段不要改能力，先把 Claude Code 与 OpenCode 在同一任务上的行为变成可比较的轨迹。Harness-Bench 和 Scaffold Effect 的共同启示是：只看最终 Pass/Fail 不够，必须看过程。[R1][R2]

| 维度   | 必须记录字段                                                               | 要回答的问题                            |
|------|----------------------------------------------------------------------|-----------------------------------|
| 模型调用 | start/end、TTFT、total latency、input/output/cache tokens、finish reason | 高延迟来自 Gateway 还是 Harness 额外 turn? |
| 探索   | grep/glob/LSP/read 次数、首次命中正确文件时间、无关文件比例                              | Claude Code 是否更快定位代码?             |
| 工具   | tool name/args/result bytes/error/retry                              | 是否存在 bash 绕过、重复工具、错误恢复差异?         |
| 上下文  | 每 turn context token、tool output 占比、compaction 时点                    | 什么时候开始膨胀? 哪类输出最浪费?                |
| 进展   | patch 数、有效新证据、重复 action、no-op turn                                   | OpenCode 是否出现 idle spinning?      |
| 验证   | test/lint/typecheck/diff review/acceptance                           | 失败是不会写，还是不会验证?                    |

首个目标不是“提高分数”，而是得到 **Failure Fingerprint**。例如：OC 的失败 40% 来自 TIME/HANG，25% 来自错误文件定位，20% 来自验证不足，只有 15% 是纯 reasoning failure。若结果真是这样，就不该优先改 Prompt 让模型“更聪明”。

## 7. P1：Turn/Stop Policy —— 对 iagent/standard 最可能产生最大 ROI

公开研究已经把 OpenCode 的一种典型失败模式归纳为 idle-loop/TIME，并观察到大量 no-action turns。  
[R1] 对你们内部高延迟 Gateway，这一问题会被放大。

**建议新增 `ProgressState`：**每个 turn 计算“是否新增了有效证据/修改/验证结果”，而不是只判断模型是否输出了文字。

```
progress_score =
  new_relevant_files * w1
+ new_evidence      * w2
+ patch_delta       * w3
+ new_test_signal   * w4
- duplicate_actions * w5
- no_op_turns       * w6
```

建议策略：连续 2-3 个 turn 没有新增证据 → 强制 replan；重复相同 tool+args → 阻止并要求说明差异；连续达到 stagnation threshold → compact/recover，而不是继续空转；完成前由 deterministic finish gate 检查 acceptance，而不是完全相信模型说“完成了”。

## 8. P1：Repository Exploration —— 目标是“更少 Token 找到更正确的代码”

OpenCode 社区已经出现模型过度使用 bash grep/find、搜索 node\_modules 或顺序读取文件导致 context 垃圾的问题反馈；官方源码 prompt 也明确鼓励 file operation 优先使用 dedicated tools。[R11][R12]

| 当前风险                        | 建议改造                                                                         |
|-----------------------------|------------------------------------------------------------------------------|
| 模型先 Read 大文件，再逐渐找目标         | 先 glob/grep/LSP，建立候选文件 Top-K，再读局部区间                                          |
| bash grep/find 绕过专用工具，输出不可控 | 文件搜索默认禁用/降低 bash 权重；必要时限制目录和输出量                                              |
| 每次搜索结果都全部塞入主上下文             | 只保留 ranking + evidence snippets；原始大结果存 trace，不进入后续上下文                        |
| 探索与实现混在同一长上下文               | 复杂任务可用一个只读 Explore subagent，主 Agent 只接收结构化摘要                                 |
| 缺乏“正确文件定位”指标                | 加入 Relevant-file Recall / Time-to-first-correct-file / irrelevant-read-ratio |

## 9. P1：Context Governor —— 这是能力、速度和稳定性的共同根因

OpenCode 的 compaction 与上下文溢出在公开 issue 中长期是高风险区域：包括 tool loop 内 overflow 检查时机、compaction 后 run 提前退出、图片/tool result 过大导致 compaction 自身失败等。[R13][R14][R15]

不要把解决方案等同于“更早 compact”。Compaction 本身也是一次昂贵模型调用，并且可能丢失决策和证据。更好的策略是先降低进入 context 的垃圾，再做结构化 compaction。

- 为每类 tool result 设置 token/byte budget：grep/list/log/test output 超限时自动摘要，同时保留可回取的 artifact id。
- 维护 `Evidence Ledger`：关键文件、符号、错误、已验证假设、当前计划作为结构化状态，避免依赖长对话回忆。
- Compaction 只压缩历史对话，不压缩 active evidence、acceptance criteria、当前 patch 状态。
- 在进入下一次模型调用前做 context budget 检查，而不是等 provider overflow 后被动补救。
- 对 iagent/standard 优先优化“prompt bytes + turn count”，因为长上下文与多轮都会叠加 Gateway 延迟。

## 10. P2：Verification Gate —— “会写代码”不等于“能完成任务”

TEBench 一类研究显示，Coding Agent 常见模式是 execute→fail→fix，但对 stale/missing tests 等需要主动语义判断的任务仍薄弱。Claude Code 的优势之一往往不是首个 patch 更漂亮，而是更稳定地走到验证闭环。

建议将完成条件从语言模型判断改为“模型 + deterministic gate”：

6. 解析用户 acceptance criteria
7. 检测 changed files 与受影响模块
8. 选择/生成 relevant tests
9. 执行 tests + lint/typecheck/build
10. 读取失败并最多定额重试
11. 最终 diff review：检查 overscope、临时代码、TODO、debug 输出
12. 只有 gate 通过才允许 finish=done

## 11. P3：为什么不建议先做 OMO 式多 Agent

你们的目标不是展示 Agent 数量，而是让 iagent/standard 的有效工作/请求比最大化。多 Agent 在宽任务上有价值，但在高延迟统一 Gateway 下，它同时增加请求数、Token、Rate Limit 压力和汇总延迟。Claude Code 目前自身也已经支持 subagents、agent teams 和 dynamic workflows，因此“有多 Agent”本身已经不构成决定性优势。[R6][R7]

| 任务类型                         | 建议                    | 原因                    |
|------------------------------|-----------------------|-----------------------|
| 简单修改/单模块 Bug                 | 单 Agent               | 协调成本大于收益              |
| 中型 Feature                   | 主 Agent + 可选 Explore  | 隔离搜索垃圾即可              |
| 复杂宽任务：多模块/Research/Test/Docs | 最多 2-3 个并行 Agent      | 并行可回本，但要有并发与 Token 预算 |
| 复杂深任务：并发 bug/架构推理            | 优先单强 Agent + reviewer | 保持连续上下文和假设链           |

## 12. 评测体系：如何证明“持平”，而不是靠主观体验

建议将验收定义为“同模型、同任务、同环境、同预算的配对实验”。Harness-Bench 的核心方法就是把 model×harness 当成评测单元。[R2]

### 四组固定条件：

- CC-IS: Claude Code + iagent/standard（主要 gold baseline）
- OC0-IS: OpenCode baseline + iagent/standard
- OC1-IS: Modified OpenCode + iagent/standard
- CC-Native: Claude Code + 原生最新 Claude/Opus（仅作为 ceiling，不参与“持平”验收）

任务集建议：第一阶段 120 个，稳定后扩到 300+ 个内部 held-out tasks。

| 类别                 | 首期数量 | 主要能力     | 典型验收                              |
|--------------------|------|----------|-----------------------------------|
| Repo Locate / Q&A  | 20   | 搜索与上下文选择 | 正确文件/符号、Time-to-first-evidence    |
| Bug Fix            | 30   | 定位+修改+验证 | 隐藏 regression tests               |
| Feature            | 25   | 多文件实现    | acceptance tests + API contract   |
| Test Evolution     | 15   | 主动验证     | missing/stale test identification |
| Refactor/Migration | 15   | 长周期一致性   | behavior-preserving hidden tests  |
| Infra/CLI/Config   | 15   | 工具与终端执行  | deterministic shell checks        |

## 13. “持平”的验收门槛：建议采用非劣效（non-inferiority）而不是追求单点领先

单次跑分波动很大。建议用 paired task design，并以配对 bootstrap / McNemar 类统计方法报告 95% CI。

| 指标                      | 建议门槛                           | 解释                               |
|-------------------------|--------------------------------|----------------------------------|
| Task Success / Pass@1   | OC1-IS 不低于 CC-IS 超过 3–5pp 的非劣界 | 主要能力门槛                           |
| Human Intervention Rate | 不高于 CC-IS                      | 不能靠更多人工救火换成功率                    |
| P50/P95 Wall Time       | P95 不超过 CC-IS 1.10×，最好更低       | iagent 高延迟下尤其重要                  |
| Model Calls / Turns     | 显著低于 OC0，目标接近或优于 CC-IS         | 核心优化指标                           |
| Input Tokens / Task     | 不高于 CC-IS 1.10×                | 避免“成功率持平、成本翻倍”                   |
| No-op/Stagnation        | 相对 OC0 至少下降 50%                | 直接对 OpenCode failure fingerprint |

|                    |            |         |
|--------------------|------------|---------|
|                    |            |         |
| Overscope / Safety | 不得劣于 CC-IS | 企业场景硬门槛 |

**为什么不用“OpenCode 分数 > Claude Code 1 分”作为成功？** 因为公开评测已经显示同一个 Harness 可能以更多 Token、更高 latency 换来几个百分点的成功率。真正的产品胜利应该是 Pareto：能力不差，同时成本、时间、干预率至少一个维度明显更好。

## 14. 如何控制“改一个地方引发连锁反应”

你的担忧是正确的：Coding Agent 是闭环系统，不是传统纯函数。Prompt 改动会改变 tool selection，tool output 会改变 context，context 又改变 compaction 与下一步行动。因此必须把“可回滚实验性”设计成一等能力。

- 所有行为改造必须有 Feature Flag，例如 `iagent\_profile\_v1`、`search\_policy\_v2`、`context\_governor\_v1`、`verify\_gate\_v1`。
- 每次只上线一个核心 hypothesis；禁止一次 PR 同时改 prompt、tool schema、compaction 和 stop policy。
- 建立 nightly regression：至少 OC0、OC1、CC-IS 三组固定版本。
- 保存完整 trajectory 与 workspace diff；一旦成功率变化，可追溯“第一次分叉 turn”。
- 先 Plugin/Hook 验证，再 Core merge。只有在 2 个以上 task family 都有稳定 Pareto 改善时才下沉。
- 版本 pin：OpenCode 上游更新后先跑兼容矩阵，不自动把 dev/latest 推给公司用户。

## 15. 建议优先修改/观察的 OpenCode 源码区域

| 区域                         | 当前关注点                                    | 第一阶段动作                                  | 风险 |
|----------------------------|------------------------------------------|-----------------------------------------|----|
| session/system.ts          | 按 model.api.id 选择模型族 prompt              | 增加显式 capability profile；确认 alias 命中     | 中  |
| session/prompt.ts          | 主 Agent loop / prompt 构建 / compaction 触发 | trace + turn budget + pre-call budget   | 高  |
| session/compaction.ts      | 摘要、overflow、continue 语义                  | 先建立 active evidence；修正边界后再重构            | 高  |
| session processor loop     | tool-result→下一 turn 的状态推进                | stagnation/no-op detector               | 高  |
| tool/grep, glob, read, lsp | repo exploration                         | 搜索优先、Top-K、输出 budget                    | 中  |
| tool/bash                  | shell 与 file ops 边界                      | 避免 bash 搜索污染；目录/输出 guard                | 中  |
| plugin hooks               | 低侵入实验入口                                  | 先实现 profiler/verification/search policy | 低  |

## 16. 推荐项目路线：6 个阶段，而不是“大重构”

| 阶段                 | 输出                                          | Go/No-Go 条件             | 核心问题       |
|--------------------|---------------------------------------------|-------------------------|------------|
| S0 基线冻结            | 固定 OC/CC/代理/model 版本 + 30 task smoke set    | 结果可重复                   | 现在到底差多少？   |
| S1 Trace           | Trajectory Profiler + failure taxonomy      | >95% run 可追踪            | 差在哪里？      |
| S2 Prompt/Profile  | iagent capability profile + prompt layering | 至少一个 family 稳定改善        | 模型是否被错误驱动？ |
| S3 Loop/Search     | stagnation + search-first + output budget   | turn/latency 显著下降且成功率不降 | 浪费在哪里？     |
| S4 Context/Verify  | evidence ledger + compaction + finish gate  | 长任务稳定性与 Pass@1 改善       | 为什么会后期失控？  |
| S5 Selective Agent | 仅宽任务引入 1-2 subagents                        | 复杂任务收益>协调成本             | 什么时候并行才值得？ |

## 17. 最终原因判断：为什么我认为“限定条件下可以持平”

- 第一，比较对象被固定成同一个模型。**这消除了最难复制的“基础模型能力差异”，把问题转化为 Harness Optimization；研究已经证明 Harness 的影响足够大。
- 第二，iagent/standard 的高 Gateway 延迟放大了 OpenCode 可优化的地方。**少一次无意义 turn、少一次大 context 请求，就直接改善 wall time；这是明确、可量化的工程收益。
- 第三，OpenCode 是开源 Harness，可以直接修改核心闭环。**Claude Code 的优势可以通过黑盒 trace 被行为分解，而不需要复制闭源实现。
- 第四，OpenCode 当前已经暴露出可定位的 failure fingerprints。**System prompt 路由、search/read 行为、context/compaction、idle-loop、tool output 等都不是“玄学差距”，而是有源码入口的工程问题。
- 第五，你们不需要在所有维度赢。**只要在公司真实 workload 上达到非劣成功率，同时把 turn/latency/cost 做到更好，就已经形成产品层面的 Pareto 优势。

## 18. 最终原因判断：为什么我仍然拒绝承诺“全面持平”

- Claude Code 原生 Claude 场景仍然是不同问题。**如果对手换成最新 Opus + Claude Code，模型-Harness 联合优化与新特性会重新拉开差距。
- 公开 Benchmark 仍不完备。**真实软件工程还包括多日演进、隐式需求、安全边界、可维护性、PR review、团队协作，这些无法被单一 SWE-bench 表示。
- Agent 系统强耦合。**任何核心改动都可能产生二阶效应，因此一次跑分提升不足以证明长期稳定。

**上游持续变化。** OpenCode 和 Claude Code 都在快速迭代；你们需要长期 regression harness，而不是一次性“赢榜”。

**iagent/standard 自身也可能变化。** 如果底层模型或 Gateway 策略升级，旧 prompt/tool tuning 可能失效，因此 capability profile 与自动回归必须成为架构的一部分。

## 19. 最终结论与建议

**结论：可以做，而且值得做；但必须把项目定义为“iagent/standard Harness 对齐工程”，不能定义为“OpenCode 全面复刻 Claude Code”。**

- 第一阶段只做 Trace + 同模型 A/B，不大改 Core。
- 第一优先级验证 iagent/standard 的 model profile / system prompt 是否正确；随后优化 turn/stop、repo exploration、context governor、verification。
- 多 Agent 放到最后，只服务于明确可并行的“宽任务”；不要用 OMO 式复杂编排掩盖单 Agent Harness 的基础问题。
- 验收采用 non-inferiority + Pareto：成功率持平只是底线，turn、P95 latency、Token、干预率至少一个维度必须更好。
- 真正的护城河不是某个 prompt，而是内部 Private Benchmark + Trajectory 数据 + 可持续 ablation 机制。

**一句话：**你们应该“测出 Claude Code 为什么在同一 iagent/standard 上更好，然后逐个消灭差异”，而不是“猜 Claude Code 有什么魔法，再把 OpenCode 改得越来越复杂”。

## 参考资料

- [R1] Vats & Golev, The Scaffold Effect in Coding Agents: Harness Choice as a Hidden Variable in Coding-Agent Evaluation (2026). <https://arxiv.org/abs/2607.22585>
- [R2] Yao et al., Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows (2026). <https://arxiv.org/abs/2605.27922>
- [R3] Zheng et al., Claw-SWE-Bench: A Benchmark for Evaluating OpenClaw-style Agent Harnesses on Coding Tasks (2026). <https://arxiv.org/abs/2606.12344>
- [R4] Artificial Analysis, AI Coding Agent Benchmarks & Harness Comparison. <https://artificialanalysis.ai/agents/coding-agents>
- [R5] SWE-bench official leaderboards. <https://www.swebench.com/>
- [R6] Claude Code Docs - Run agents in parallel. <https://code.claude.com/docs/en/agents>
- [R7] Claude Code Docs - Agent Teams / Dynamic Workflows. <https://code.claude.com/docs/en/agent-teams>
- [R8] Claude Code Docs - Context Window / Compaction. <https://code.claude.com/docs/en/context-window>
- [R9] OpenCode source - packages/opencode/src/session/system.ts. <https://github.com/anomalyco/opencode/blob/dev/packages/opencode/src/session/system.ts>
- [R10] OpenCode issue - agent.prompt model/auth path inconsistency. <https://github.com/anomalyco/opencode/issues/13714>
- [R11] OpenCode issue - unwanted grep/find usage. <https://github.com/anomalyco/opencode/issues/6506>
- [R12] OpenCode Anthropic prompt - prefer dedicated tools over bash. <https://github.com/anomalyco/opencode/blob/dev/packages/opencode/src/session/prompt/anthropic.txt>
- [R13] OpenCode issue - compaction exits run unexpectedly. <https://github.com/anomalyco/opencode/issues/13946>
- [R14] OpenCode issue - compaction overflow / large tool output. <https://github.com/anomalyco/opencode/issues/10634>
- [R15] OpenCode issue - oversized session cannot compact. <https://github.com/anomalyco/opencode/issues/17340>
- [R16] OpenCode Docs - Plugins. <https://opencode.ai/docs/plugins/>
- [R17] Claude Code Docs - How Claude Code works. <https://code.claude.com/docs/en/how-claude-code-works>

**内部数据说明：**报告中关于 iagent/standard 的映射、IAgent Gateway 协议路径及延迟/TPS 的描述来自此前已提供的内部测试信息，不属于公开论文数据；正式立项时建议将这些测量重新固化为可复现基线。